



FINE JEWELLERY · FRONT-END DOCUMENTATION

OPALÉRA

The House of Every Gem

How the jewellery_Html.html storefront works:
structure, design system, animation, data and interactivity

QUANTUM CODERS · INFORMATICS 2B
80 PIECES · 3 COLLECTIONS · ONE SINGLE FILE
29 JULY 2026

DOCUMENT

This document explains the front-end page `jewellery_Html.html`: what appears on screen, how the visual design was chosen, how the animated background works, where the data and the product photographs come from, and how the interactive features are implemented. It is written so that any member of the team can read the page's source code with confidence and extend it.

01 Overview

OPALÉRA is a single-file storefront for the Jewellery Dataset. The entire application — markup, styling, data and behaviour — lives in one HTML file with no frameworks, no build step and no server: opening the file in any modern browser is enough. The store presents all 80 pieces from `Jewellery_Dataset.csv` across three collections (necklaces, earrings and pendants), together with a small data-visualisation section that summarises the dataset graphically.

The name OPALÉRA derives from the opal, the one gemstone whose internal fire flashes the colours of every other gem — sapphire blue, emerald green, rose blush, diamond white and golden amber. Since the dataset spans exactly that spectrum of stones and metals, the opal stands as a symbol for the whole collection; the page's colour system is built from the same idea.

Key facts

PROPERTY	VALUE
File	<code>jewellery_Html.html</code> (single, self-contained page)
Data source	<code>Jewellery_Dataset.csv</code> — 80 rows embedded directly in the page as a JavaScript array
Collections	Necklaces (30) · Earrings (20) · Pendants (30)
Images	Hot-linked from the dataset's original public archive on GitHub; local <code>img/</code> folder kept as offline fallback
Technologies	HTML5, CSS3 (animations, grid, backdrop-filter), vanilla JavaScript (canvas, IntersectionObserver)
Dependencies	None — no framework, no CDN library, no build step

02 Page Anatomy

The page is a vertical composition of six regions, with a product-detail modal floating above them. Two decorative layers (the aurora and the sparkle canvas) sit behind everything and never intercept mouse input.



- **Navigation** — A fixed translucent bar with the rotated-diamond brand mark (its core pulses on a 3-second loop) and quick links that pre-filter the collection. It blurs whatever scrolls beneath it (backdrop-filter).
- **Hero** — A full-viewport landing. The store name is rendered at up to 7.5rem with a golden sheen that sweeps across the letters every 6 seconds; four outlined gem glyphs float around the heading on offset 9-second cycles.
- **The Atelier (data section)** — Four stat tiles whose numbers count up from zero when scrolled into view, plus two bar charts — pieces per collection and average price per collection — whose bars grow when first seen. All figures are computed live from the embedded dataset.
- **The Collection** — A responsive card grid (auto-fill, minimum 255px per column). Each card carries the photograph, collection chip, name, price in rand and a hover light-sweep.
- **Modal** — Clicking a card opens the piece’s full record: large photo, collection, price, the craftsmanship specification and the marketing description from the CSV. It closes on the × button, a click outside the sheet, or the Escape key.
- **Footer** — The maison signature and a provenance note stating that imagery is streamed from the dataset’s original archive.

03 Design System

Colour palette

The palette is defined once as CSS custom properties on :root, so re-theming the entire page means editing a handful of variables. A near-black violet base makes the gold read as metal rather than yellow; the four gem hues are reserved for the aurora, the charts and small accents.

	--bg	#0b0a10	page base — near-black violet
	--bg-2	#14121d	panels, table headers

	--card	#171522	product cards
	--gold	#d8b45a	primary accent, buttons, rules
	--gold-light	#f0d896	prices, highlights, title sheen
	--champagne	#e9dcc0	display headings
	--sapphire	#3f6fd8	aurora / chart hue
	--emerald	#2fae7e	aurora / chart hue
	--rose	#e08bb0	aurora / chart hue
	--amethyst	#9b6fd8	aurora hue

Typography

Two families carry the whole page. Georgia (serif) sets everything readable — the display name, product names, prices and body copy — giving the maison its editorial voice. Arial (sans-serif) is used only at small sizes for uppercase micro-labels with wide letter-spacing (0.26–0.5em): eyebrows, tabs, chips and buttons. This serif-display / sans-label pairing is borrowed from the Pro Tech AI site’s editorial style, as are the numbered section headings (01, 02).

Motion inventory

ELEMENT	ANIMATION	DURATION
Aurora blobs (4)	drift — translate and scale, alternating	26 s loop
Sparkle field	canvas particles rise and twinkle (requestAnimationFrame)	continuous
Store name	sheen — gradient swept through background-clipped text	6 s loop
Brand mark core	gemPulse — scale and fade	3 s loop
Hero gems (4)	floatGem — rise and tilt, staggered delays	9 s loop
Scroll hint	bob — gentle vertical nudge	2.4 s loop
Product cards	entrance fade-and-rise, staggered 40 ms per card	on render
Card hover	lift, image zoom (1.09×), light sweep across photo	on hover
Stat numbers	count-up with cubic ease-out	1.4 s, on first view
Chart bars	width grows to value	1.4 s, on first view
Modal sheet	rise — fade, lift and settle	0.5 s, on open

04 The Animated Background

The requested “background movement” is produced by two independent, always-running layers that sit behind the content (z-index 0 and 1, with all content wrapped at z-index 2). Both are purely decorative: pointer-events are disabled so they can never block a click.

Layer 1 — the aurora

Four large radial-gradient circles — sapphire, amethyst, emerald and rose — are blurred by 110px and drifted slowly around the viewport by one shared CSS keyframe animation (26 seconds, alternating direction, staggered with negative delays so they never move in unison). At 30% opacity over the near-black base they read as faint coloured light moving through a dark room — the opal’s fire, page-sized.

```
.aurora span { position:absolute; border-radius:50%; filter:blur(110px); opacity:.30;
  animation: drift 26s ease-in-out infinite alternate; }
@keyframes drift { 0% {transform:translate(0,0) scale(1)}
  50% {transform:translate(6vw,-4vh) scale(1.15)}
  100%{transform:translate(-5vw,5vh) scale(.92)} }
```

Layer 2 — the sparkle field

A full-screen `<canvas>` hosts up to 110 tiny particles (the count scales with viewport width). Each particle carries its own radius, rise speed, twinkle phase and one of four gem hues (gold, sapphire, rose, emerald). Every animation frame the particle rises, its opacity oscillates through a sine wave (the twinkle), and when it leaves the top edge it re-enters at the bottom at a random horizontal position. The loop uses `requestAnimationFrame`, so it pauses automatically in background tabs, and it is skipped entirely when the visitor’s system asks for reduced motion.

Respecting reduced motion

A prefers-reduced-motion media query disables every CSS animation and transition, and the same check in JavaScript stops the canvas loop after one static frame. Visitors who opt out of motion see a calm, fully functional page.

05 The Data Layer

The 80 rows of `Jewellery_Dataset.csv` are embedded in the page as a compact JavaScript array of `[id, category, name, price, image]` tuples, then expanded into objects. Because the two long CSV text columns (description and detailed_description) are identical for every row of a category, they are stored once per collection in the SPEC and DESC lookup tables — the modal recombines them at view time. This keeps the file small without losing any information from the dataset.

```
const P = [ [1,"necklace","Sapphire Necklace",1499,"img/necklace_88.jpg"], ... 80
rows ];
const SPEC = { necklace:"Amulette de Cartier necklace, XS model, ...", earrings:...,
pendant:... };
const DESC = { necklace:"Enhance your elegance with our exquisite ...", ... };
```

Every figure shown in the Atelier section is computed from this array at load time — nothing is hard-coded. Adding a row to `P` automatically updates the tab counts, the stat tiles and both charts.

What the data shows

COLLECTION	PIECES	AVERAGE PRICE	PRICE RANGE
Necklaces	30	R 1 939	R 1 299 – R 2 899
Earrings	20	R 904	R 599 – R 1 599
Pendants	30	R 751	R 299 – R 999
All pieces	80	R 1 235	R 299 – R 2 899

The two bar charts translate the first two columns of this table into animated horizontal bars, coloured gold, sapphire and rose to match the collections' accents. Prices are formatted as South African rand with `toLocaleString("en-ZA")`.

06 Interactive Features

- **Filtering** — Four tabs (All · Necklaces · Earrings · Pendants) each display their live count. A click sets the filter state and re-renders the grid; the nav links reuse the same `setFilter()` call, so “Earrings” in the menu both scrolls to the collection and filters it.
- **Search** — The search box filters by product name on every keystroke, combined with the active tab (an AND, not an OR). An empty result shows a courteous message instead of a blank grid.
- **Sorting** — A select offers curated order (dataset order), price ascending, price descending, and name A–Z. Sorting copies the array before ordering it, so the curated order is never destroyed.
- **Product modal** — `openModal(p)` fills the sheet from the clicked product and locks page scroll; `closeModal()` releases it. Escape, the close button and a click on the dark backdrop all close it.
- **Scroll-triggered reveals** — One `IntersectionObserver` watches the stat tiles and chart cards. The first time 30% of an element is visible, its counters ease from 0 to their target over 1.4 seconds and its bars grow to width — then the observer lets go, so the effect plays exactly once.
- **Sticky toolbar** — The filter bar sticks below the nav while the visitor browses the grid, so the controls are always within reach on a long page.

07 Image Delivery — the Real Pictures, No Downloads

The photographs are not stored with the page. Research traced `Jewellery_Dataset.csv` to its original public source: the GitHub repository `KaushalBEHRA/khanak` (“An ecommerce Jewellery Site”), whose `Khanak Database.csv` is byte-identical to the course’s CSV and whose `img/` folder holds every one of the 80 referenced photographs. The page hot-links those originals directly.

```
const CDN = "https://raw.githubusercontent.com/KaushalBEHRA/khanak/0ba03099d1074a13924b1e40ce35ab10571e7bce/img/";
const webSrc = p => CDN + p.img.split("/").pop();
```

- **Pinned to a commit** — The URL embeds a full commit hash rather than a branch name, so the links can never silently change or break if the repository moves on — the images are frozen at a known-good state.
- **Verified end-to-end** — All 80 URLs were checked twice: HEAD requests confirmed every one answers HTTP 200 with content-type `image/jpeg`, and the live page was then instrumented to load all 80 in the browser — 80 of 80 loaded, none failed.
- **Correctness bonus** — The archive’s images match their CSV rows, whereas several files in the local `img/` folder do not (the local `necklace_88.jpg`, for instance, is a photograph of earrings). Linking the originals therefore also corrected the catalogue.

The fallback chain



The chain is implemented in one small function: each `<img>` carries its web URL as `src` and its local path in a `data-local` attribute; the first error swaps to the local path, a second error dims the image. The result is a page that works anywhere with internet, still works offline next to the `img/` folder, and degrades politely if both fail.

08 Code Reference

FUNCTION / BLOCK	RESPONSIBILITY
products (from P)	The dataset: 80 tuples expanded to {id, category, name, price, img} objects.
render()	Applies filter + search + sort, rebuilds the card grid, staggers the entrance animation, wires each card to the modal.
setFilter(key)	Sets the active collection, synchronises tab styling, re-renders. Also called by the nav links.
fmt(n)	Formats a number as South African rand (R 2 499).
openModal(p) / closeModal()	Fill and show / hide the product sheet; lock and release page scroll; Escape-key handler.
webSrc(p) / imgFallback(el)	Build the pinned archive URL for a product; walk the web !' local !' dim fallback chain on error.
bars(...) / avg(...)	Compute per-collection statistics and build the two bar charts from them.
IntersectionObserver block	Plays the count-up and bar-growth animations once, on first visibility.
Sparkle canvas block	Sizes the canvas, seeds the particles, runs the <code>requestAnimationFrame</code> twinkle loop, honours reduced motion.

How to customise

- **Rename the store** — Change the `<h1>`, the nav brand text and the `<title>`; the sheen and layout adapt automatically.
- **Re-theme** — Edit the CSS variables in `:root` — every colour on the page, including charts and buttons, follows them.
- **Add a product** — Append one line to the `P` array. Tabs, counters, charts and the grid all update themselves.
- **Change currency** — Edit `fmt()` — one line — e.g. replace `"R"` with `"$"` or switch the locale.

09 Performance & Accessibility

- **Lazy images** — Card photographs use `loading="lazy"`, so the browser fetches them only as the visitor approaches them — the page is interactive long before all 80 images arrive.
- **No dependencies** — Zero frameworks and zero external scripts: the only network traffic is the photographs themselves.
- **Cheap animation** — The heavy motion runs on GPU-friendly properties (transform, opacity) and one small canvas; particles are capped and scale down on narrow screens.
- **Reduced motion** — Both CSS and the canvas loop honour prefers-reduced-motion (see section 04).
- **Semantics & keyboard** — Landmark elements (nav, header, section, article, footer), alt text on every image, and Escape closes the modal.
- **Responsive** — The grid auto-fills from four columns on desktop down to one on a phone; the nav collapses its link row; the modal stacks its two panes below 720px.

OPALÉRA — eighty pieces, one file, and a background that never stops shimmering.